

Leant2: Design for Further Improvements

Engineering plan, open algorithmic problems,
and where outside expertise would help

Prepared for Vladimir Reshetnikov

September 18, 2026

Abstract

Leant2 now exists as a working Lean 4 program. It passes the owner’s minimal baseline, the Leant `:synth` golden corpus (278 of 278 queries by outcome), and every other Leant suite that was replayed against it: the recursive scenarios, the Church behavior probes, the contextual and behavioral suites, the 350-signature Church corpus and the session suite. All of it also passes with the per-query budget halved. This document is about what comes next. It starts from measurements rather than from the earlier proposals: a probe of sixteen queries just outside the corpus shows exactly where the engine stops (recursion on natural numbers, conditionals, induction over indexed families, programs that combine two recursive results, library lemmas), and a profile of the search shows where its time goes.

The document separates three kinds of work. *Engineering* items have a known solution and only need to be built. *Adaptation* items are solved in the literature for simply typed or ML-like languages and must be re-derived for Lean’s dependent type theory and its kernel. *Research* items are those where, to our knowledge, no optimal or even clearly adequate algorithm is known. Nine research problems are stated in full: search control under a wall-clock budget with coupled goals; invention of carriers, accumulators and auxiliary state for folds and recursors; evaluation of programs with holes; synthesis over indexed families and the motive problem; general recursion from incomplete examples; contracts beyond decidable observations and the co-synthesis of programs with inductive proofs; component retrieval at library scale; ranking and candidate equivalence; and learned guidance under a kernel gate. For each, the document gives the problem, why it is hard, what is known, the design we would try first, how we would measure progress, and the precise questions we would put to outside experts, together with the projects and fields whose research bears on them.

Status. This is a design document. Every number in Section 1 was measured on the repository at commit `33cec8b` with Lean 4.34.0 on one Windows workstation. Nothing else in the document is measured: the designs in Parts II and III are proposals, not results. Literature is cited from memory of the published record; titles, venues and years should be checked before they are relied on in print.

Contents

I	Where the engine stands	5
1	Evidence	5
1.1	What passes	5
1.2	What does not pass	5
1.3	Where the time goes	6
1.4	What the search is	7
2	Constraints and classification	7
2.1	Constraints that stay	7
2.2	Three classes of work	7
2.3	Work items	8
II	Work with known solutions	9
3	Engineering items	9
3.1	E1. Results as compilable definitions	9
3.2	E2. Frontends	9
3.3	E3. Receipts, replay and resumable search	10
3.4	E4. Substrate cost	10
3.5	E5. Parallel lanes	10
3.6	E6. A decision procedure for propositional targets	11
3.7	E7. A wider proof portfolio	11
3.8	E8. Benchmarks beyond Leant	11
4	Adaptation items	12
4.1	A1. Conditionals and guard abduction	12
4.2	A2. Recursion on natural numbers, literals and arithmetic	12
4.3	A3. Normal forms and symmetry reduction	13
4.4	A4. Parametricity-directed test inputs	13
4.5	A5. Rule indexing	14
III	Open problems	14
5	R1. Search control with coupled goals under a wall-clock budget	14
5.1	The problem	14
5.2	Why it is hard here	14
5.3	What is known	15

5.4	Design to try first	15
5.5	What would count as progress	16
5.6	Questions for experts	16
6	R2. Carriers, accumulators and auxiliary state	16
6.1	The problem	17
6.2	What is known	17
6.3	Design to try first	18
6.4	What would count as progress	18
6.5	Questions for experts	19
7	R3. Evaluation of programs with holes	19
7.1	The problem	19
7.2	What is known	20
7.3	Why it is hard here	20
7.4	Design to try first	21
7.5	Soundness conditions	21
7.6	Questions for experts	21
8	R4. Indexed families and the motive problem	22
8.1	The problem	22
8.2	What is known	22
8.3	Why it is hard	23
8.4	Design to try first	23
8.5	Questions for experts	23
9	R5. General recursion from incomplete examples	24
9.1	The problem	24
9.2	What is known	24
9.3	Why it is hard here	24
9.4	Design to try first	25
9.5	What would count as progress	25
9.6	Questions for experts	25
10	R6. Contracts beyond decidable observations	26
10.1	The problem	26
10.2	What is known	26
10.3	Why it is hard	27
10.4	Design to try first	27
10.5	Questions for experts	27
11	R7. Component retrieval at library scale	28

11.1 The problem	28
11.2 What is known	28
11.3 Why it is hard here	28
11.4 Design to try first	29
11.5 Questions for experts	29
12 R8. Ranking and candidate equivalence	29
12.1 The problem	29
12.2 What is known	30
12.3 Design to try first	30
12.4 Questions for experts	30
13 R9. Learned guidance under a kernel gate	31
13.1 The problem	31
13.2 What is known	31
13.3 Design to try first	31
13.4 Questions for experts	31
IV Plan	31
14 Roadmap	32
14.1 Dependencies	32
14.2 Phases and gates	32
14.3 Risks	33
15 Where outside expertise would help	33

Part I

Where the engine stands

1 Evidence

This section records what was measured. Everything later in the document is argued from it.

1.1 What passes

Table 1 lists the acceptance harnesses in `tools/`. Each replays a Leant suite through the Leant2 REPL. Leant ran most of them once per engine (Djinn, Exference, Both); Leant2 has one adaptive engine and no settings, so each case runs once. A case is scored by outcome category, not by transcript text.

Harness	Source in Leant	Score	Wall time
<code>run_baseline.py</code>	<code>test/synth-*.txt</code> , 30 transcripts	278/278	109 s
<code>run_recursive.py</code>	<code>test-recursive</code>	9/9	5 s
<code>run_church.py</code>	<code>test-church</code> behavior probes (core, extended, partial)	28/28	162 s
<code>run_context.py</code>	<code>test-behavioral</code> , <code>test-context</code> , nested forall	95/95	8 s
<code>run_corpus.py</code>	<code>test-church</code> signature corpus	350/350	11 s
<code>run_session.py</code>	session provider identity	6/6	19 s

Table 1: Acceptance harnesses at commit 33cec8b, 10 s budget per query. The same scores hold with a 5 s budget. The Church wall time is dominated by thirteen unscored stretch cases that each run to the budget.

After the last round of work the slowest scored case takes 2.0 s (Church `foldl`); the next slowest take 1.1 s and 0.7 s. Most contract queries finish in the 400 ms grace period that follows the first accepted candidate.

1.2 What does not pass

Two groups of cases fail.

Stretch cases. Thirteen of the nineteen partial Church operations (`foldl1`, `foldr1`, `reduce`, `maximum`, `maximumBy`, `minMax`, `at` and their variants) are not solved within 10 s, and four that were retried are not solved within 60 s either. Leant’s own behavior ledger records every one of them as failed or unresolved in all engines, so this is parity, not regression. Their reference solutions need a fold whose carrier holds more than the result: an optional first element, a pair of running extremes, or an index together with a continuation.

A probe just outside the corpus. Sixteen small queries were written to find the edge of the engine (Table 2). They are ordinary first exercises in the program-synthesis literature. Seven fail, and one succeeds with an error.

#	Query	Outcome	Reason
1	reverse a <code>List</code> , two examples	found, 0.4 s	library provider <code>List.reverse</code>
2	append two lists	found	provider
3	double a <code>Nat</code>	found	<code>Nat.add a a</code>
4	2^n on <code>Nat</code> , three examples	rejected	no recursion on <code>Nat</code> ; 3675 programs refuted
5	length of a list	found	provider
6	sum of a <code>List Nat</code>	found, error	<code>List.rec</code> candidate; binding it fails in the code generator
7	head of <code>Vec A (n+1)</code>	found	dependent case analysis works
8	map over <code>Vec A n</code>	not found	induction with an index-dependent motive
9	maximum of two <code>Nat</code>	rejected	no conditionals or comparisons
10	drop zeros from a <code>List Nat</code>	rejected	conditional inside a fold
11	in-order flatten of a binary tree	rejected	combines two recursive results; depth
12	quantified length-preservation contract	found	identity; proved by <code>simp</code>
13	$\exists n, P n$ from $P 3$	found	witness from a local
14	an element of <code>Fin (n+1)</code>	timeout	needs a library lemma ($0 < n + 1$)
15	symmetry of equality	found	
16	transitivity of $\leq$ on <code>Nat</code>	not found	library lemma, no retrieval

Table 2: Edge probe, 6 s budget. “Rejected” means programs of the type were proposed and every one failed the contract.

1.3 Where the time goes

The lane trace (`set_option leant2.trace true`) reports self times of the search steps. On the slowest query before the last optimization, the Church `maybeEither` probe (5.4 s, 2858 search nodes), the breakdown was:

Step	Time	Share
Residual evaluation of the contract on partial programs	2.4 s	45%
of which kernel reduction	1.8 s	
of which substituting delayed assignments	0.4 s	
Deciding the contract on closed programs	0.7 s	13%
Forward-application preparation (rule 7b)	0.2 s	4%
Unification in exact-local and apply steps	0.2 s	4%
Save and restore of the metavariable context	0.07 s	1%
Everything else instrumented	0.2 s	4%
Not attributed by any timer	1.7 s	30%

Table 3: Self times on one contract-heavy query. Roughly 1.9 ms per search node.

Three facts from this profile matter later.

1. **One rule was 90% of the search.** Disabling the accumulator-instantiation rule (7a) alone took the query from 5.4 s to 0.5 s. The rule was then restricted to recursive eliminators. Rule cost is dominated by the subtree a rule opens, not by the rule’s own time, and we have no model that predicts it.
2. **Evaluation dominates per-node cost** whenever a contract is present, even after three optimizations (deciders synthesized once per query, kernel instead of meta-level reduction, skipping re-evaluation while it is stuck on the same holes).

3. **A third of the time is unexplained.** Save and restore are cheap, so the usual suspect is cleared. The remainder is spread over many small monadic operations per node.

1.4 What the search is

For the rest of the document it helps to state the search precisely. A state is a Lean metavariable context together with an ordered list of open goals, each carrying a remaining depth, a deferral count and a set of consumed eliminators. A step takes the first goal, tries an ordered list of about fifteen rules, and for each alternative saves the context, applies the rule, and continues with the rule's subgoals prepended to the remaining goals. Failure restores the context. The order of goals is changed by deferral rules. The whole tree is searched depth first with iterative deepening on the depth bound, under lane deadlines, and the first accepted candidate starts a 400 ms grace period. There is no memoization across branches, no cost model beyond depth, and no learning within or across queries.

This design was chosen because goals share metavariables: assigning a hole in one goal changes the type of another, so goals cannot be solved independently. Carrying all unfinished obligations in the continuation makes backtracking correct by construction. Its price is that the search order is fixed by the rule order and the goal order, and every improvement in Part III has to live with that or replace it.

2 Constraints and classification

2.1 Constraints that stay

The owner's requirements and the decisions of the unified proposal are not reopened here.

- **One adaptive engine, no settings.** Every improvement must decide for itself when it applies. A technique that needs the user to choose a mode is out of scope; a technique that needs a tuning constant must come with a way to set the constant from the query.
- **The kernel is the only authority.** A candidate is published only after the exact program and the exact contract proof pass `addDeclCore` and the axiom audit. Heuristics, evaluators, learned models and external tools may propose and may prune; they never certify.
- **Pruning must be sound with respect to the contract.** A branch may be abandoned only when no completion of the partial program can satisfy the contract, or when the loss is an accepted incompleteness of a bounded search (depth, splits). The first kind needs an argument; Section 7 returns to this.
- **The baseline never regresses.** The six harnesses of Table 1 are the regression gate for every change, at 10 s and at 5 s.
- **Core-only by default.** The toolchain is Lean 4.34.0 without Mathlib. Mathlib, Aesop, Plausible and similar packages may become optional lanes, never requirements.
- **Determinism.** The same query in the same environment gives the same candidates in the same order. Parallelism and learned guidance have to preserve this, or make the nondeterminism invisible in the result.

2.2 Three classes of work

[E] Engineering. The algorithm is known and has been used in a comparable setting. The risk is effort and integration.

[A] Adaptation. The algorithm is known for simply typed or ML-like languages, or inside Lean for proofs rather than programs. It must be re-derived for dependent types, universe

polymorphism, type classes, and a search whose state is a metavariable context. The risk is that the adaptation loses the property that made the original work.

[R] Research. We do not know of an algorithm that is optimal, or even clearly adequate, for the problem as it arises here. These are the places where time spent reading, experimenting and talking to specialists is likely to pay more than time spent coding.

2.3 Work items

Id	Item	Class	Evidence that motivates it
E1	Emit results as compilable definitions	E	probe 6: <code>List.rec</code> fails in the code generator
E2	Frontends: term, tactic, code action, sketches with holes	E	only a REPL and <code>#leant2</code> exist
E3	Receipts, replay and resumable search	E	unified proposal, not built
E4	Substrate cost: the unattributed 30%	E	Table 3
E5	Parallel lanes	E	lanes run one after another
E6	Decision procedure for intuitionistic propositional targets	E	negative results are bounded-search only
E7	Wider proof portfolio (<code>grind</code> , bounded instantiation)	E	quantified contracts rely on <code>simp</code>
E8	Benchmark suite beyond Leant	E	corpus saturated at 100%
A1	Conditionals and guard abduction	A	probes 9, 10
A2	Recursion on <code>Nat</code> , literals and arithmetic components	A	probe 4
A3	Normal forms and symmetry reduction in enumeration	A	junk candidates in traces
A4	Parametricity-directed test inputs	A	contracts are user-written examples only
A5	Rule indexing with discrimination trees	A	every rule scans every local
R1	Search control with coupled goals under a wall-clock budget	R	one rule was 90% of a search
R2	Carriers, accumulators and auxiliary state	R	13 stretch cases; probe 11
R3	Evaluation of programs with holes	R	45% of time on contract queries
R4	Indexed families and the motive problem	R	probe 8
R5	General recursion from incomplete examples	R	probes 4, 11
R6	Contracts beyond decidable observations; program and proof together	R	portfolio is five tactics
R7	Component retrieval at library scale	R	probes 14, 16; twenty curated providers
R8	Ranking and candidate equivalence	R	three-key lexicographic ranking
R9	Learned guidance under a kernel gate	R	none today

The research items are not independent. R2 and R5 are the same problem seen from Church encodings and from recursors: both ask for a type that the specification does not mention. R3 is what makes R2 and R5 affordable, because a carrier is only worth trying if wrong ones are refuted early. R1 decides how much of any of the others fits in ten seconds. Section 14 orders the work accordingly.

Part II

Work with known solutions

3 Engineering items

Each item names the defect, the design, and the test that closes it.

3.1 E1. Results as compilable definitions

Defect. The REPL binds every candidate as `it1`, `it2`, ... with `addAndCompile`. A candidate built from a recursor, such as the list sum

```
fun a => List.rec Nat.zero (fun head tail tail_ih => head.add tail_ih) a
```

is accepted by the kernel and then rejected by the code generator (“does not support recursor `List.rec`”). The user sees a correct answer followed by three errors, and cannot evaluate the result.

Design. Separate the certified term from the published definition.

1. The certified object stays the kernel term. Bind it with `addDecl` only, so that binding cannot fail after acceptance.
2. Produce a *presentation*: translate each recursor application with a constant motive into a structurally recursive `def` with pattern matching, elaborate that definition with Lean’s own equation compiler, and prove it equal to the certified term by `rfl` (or by induction when `rfl` fails). When the proof succeeds, attach the compiled definition with `@[implemented_by]` or publish it in place of the recursor term; when it fails, publish the kernel term as `noncomputable` and say so.
3. The same translation gives the text shown to the user: a `match` is what a Lean programmer expects to read, and `casesOn` terms in the output become `match` expressions too.

The translation is mechanical for constant motives and non-indexed families, which is all the engine produces today. Non-constant motives (Section 8) need the translation to generalize over the motive’s extra arguments; Lean’s structural-recursion elaborator already does the reverse direction and its source is the specification.

Closing test. Probe 6 binds without error and `#eval it1 [1,2,3]` prints 6; every recursive candidate in `run_recursive.py` is printed as a `match`.

3.2 E2. Frontends

Today there are two entry points: the REPL and the command `#leant2`. The engine entry `runQuery` already takes a closed type, an optional contract, a profile and a provider list, so the frontends are thin.

Term elaborator `synth%` elaborates to the first-ranked candidate for the expected type, with the local context as providers. It must postpone until the expected type has no metavariables, and it must fail rather than guess when the budget ends.

Tactic `leant2` closes a goal whose target is data or a proposition, reporting the term through `Try this`. It competes with `exact?`; its advantage is construction (case analysis, recursion, contracts) rather than lookup.

Sketches A definition body with holes, `def f xs := List.foldr ?step ?init xs`, plus a `where` contract, is a query whose root program is partially given. The search state already represents exactly this, so the work is in elaborating the sketch with `?`-holes into the initial goal list. Sketches are also how a user passes a carrier hint without a setting.

Code action In the editor, offer “synthesize body” on a `sorry` whose enclosing declaration has example-style tests below it; collect the tests as the contract.

Library API A documented `Leant2.synthesize : Query -> MetaM Outcome` with the result algebra, for use from other metaprograms, including Forge.

Closing test. Each frontend has a test file under `tests/`; the sketch frontend re-solves the thirteen stretch cases when given the reference carrier, which also measures how much of their difficulty is carrier invention alone.

3.3 E3. Receipts, replay and resumable search

The unified proposal specifies a receipt for every accepted candidate (query, environment fingerprint, program, proof, axioms, ledger) and a replay command that re-checks a receipt without search. Neither is built. Build them as specified; the gate already has every field. Resumability is the new part: a `budgetExhausted` outcome should carry enough to continue. With iterative deepening the resumable state is small, namely the lane, the depth, and the path of alternative indices of the node that was being expanded when the deadline hit. Replaying that path re-creates the metavariable context deterministically, which is cheaper and more robust than serializing it.

Closing test. A query cut at 2 s and resumed twice finds the same candidates in the same order as one run at 6 s.

3.4 E4. Substrate cost

Thirty percent of the time on contract queries is not attributed to any step (Table 3). The plan is measurement first.

1. Build the executable with a sampling profiler attached (on Windows, ETW; on Linux, `perf`) and read the C call stacks of the Lean runtime. This distinguishes allocation, reference counting, persistent-map operations and `instantiateMVars` traversals without guessing.
2. Remove repeated work per node that is already visible in the source: the local context is scanned by six rules, each recomputing `whnf` of every local’s type; goal types are instantiated at entry, in the rotation check, and again in the residual scan; `instantiatePartial` rebuilds the whole program at every evaluation. Compute per-local facts once when a local is introduced, and keep them in the goal record.
3. Replace linear scans by indexes (item A5).

Closing test. Unattributed time under 10%; node cost under 1 ms on the Church probes.

3.5 E5. Parallel lanes

The constructive, refutation and classical lanes are independent searches over the same frozen environment. Lean can run `MetaM` computations in tasks when each has its own state; the lanes share only the ledger and the first-found signal, which are already `IO.Refs`. Run lanes as tasks with a common deadline and cancel the others when one reports. Determinism is kept by ranking after all lanes have stopped and by fixing a priority between lanes when more than one succeeds.

Or-parallelism inside a lane (the alternatives at the root, or the carriers of Section 6) follows the same pattern and matters more once carrier enumeration exists. The limit is memory, since each task holds its own caches.

Closing test. Uninhabited and classical queries in the baseline finish in the time of their own lane rather than after the constructive lane’s share.

3.6 E6. A decision procedure for propositional targets

For a target in the implicational-conjunctive-disjunctive fragment over propositional atoms, intuitionistic provability is decidable, and contraction-free sequent calculi of the LJT family terminate without loop checking [11]. The unified proposal already specifies a reification and a focused calculus for this fragment. Building it gives three things the bounded search cannot: a definite “not intuitionistically provable” with a finite Kripke countermodel as evidence, a definite answer for classical provability (by a SAT check on the same reification), and therefore a principled decision to enter the classical lane instead of the present test for the presence of **Prop**. The procedure is PSPACE-complete in general [29] but the instances are tiny.

Closing test. Every propositional query in **synth-basic** and **synth-manual** is answered by the procedure with a certificate, positive or negative, in under 50 ms.

3.7 E7. A wider proof portfolio

Closed non-decidable conjuncts go to **first** | **rfl** | **decide** | **simp** | **omega**. The toolchain also ships **grind** and **try?**, and the unified proposal asks for a proof-service adapter with typed replies and per-tier budgets. Build the adapter, order the tiers by measured cost, add **grind** and **simp_arith**, and add bounded instantiation: for a contract $\forall x : \tau, P x$ with τ finite or with a small-scope hypothesis, test P on the first k values before any proof attempt, so that a wrong candidate is refuted by evaluation and only survivors reach the prover. The refutation is sound; the proof is still required for acceptance.

Closing test. A suite of twenty quantified contracts over lists and naturals (length preservation, idempotence, involution) where wrong candidates are refuted by instantiation in under 5 ms each.

3.8 E8. Benchmarks beyond Leant

The Leant corpus is saturated. Progress on Part III needs problems that fail today and have published baselines. Port, with their examples as contracts: the benchmark sets of Myth [24], λ^2 [19], Smyth [28], Burst [5] and Trio [34] (lists, naturals, trees, booleans; about fifty problems each with heavy overlap), the type-only problems used to evaluate Agda’s and Coq’s inhabitation tools [3, 26], and a Lean-specific set made by erasing the bodies of small definitions in core **List**, **Option** and **Nat** and using their **simp** lemmas as contracts. Record solved count, time to first candidate, and rank of the reference solution. The thirteen stretch cases and the seven failing probes of Table 2 are the first entries.

4 Adaptation items

These are solved problems in other settings. For each we say what carries over and what does not.

4.1 A1. Conditionals and guard abduction

Gap. The engine cannot produce `if a <= b then b else a`. It has no comparison components and no rule that introduces a Boolean or decidable test. Probes 9 and 10 fail for this reason alone.

What is known. Enumerative synthesizers separate *terms* from *conditions*: find terms that are each correct on some examples, then learn a decision tree of guards that routes every example to a correct term (the divide-and-conquer strategy of EUSolver [13]). Type-directed synthesizers abduce the guard instead: Synquid generates a branch, asks a solver for the weakest condition under which it satisfies the refinement type, and synthesizes a guard that implies it [30]. Example-directed tools split on a scrutinee when the examples disagree about the result [24].

Adaptation. The engine already evaluates the contract on partial programs. Extend the residual evaluator to report, for a closed branch candidate t , the *set of observations* it satisfies, instead of only refuting at the first failure. Then:

1. collect branch candidates whose satisfied sets cover all observations;
2. enumerate guards from a small grammar over the locals (decidable comparisons of locals of the same type, Boolean-valued locals and their applications, `isEmpty`-style tests obtained from constructors);
3. evaluate each guard on each observation's inputs, and build the smallest decision tree that separates the satisfied sets;
4. emit `if g then t1 else t2` (or `match`), and let the ordinary closed-program check certify it.

The part that does not carry over is identifying “the inputs of an observation”. In a SyGuS problem an example is an input–output pair. Here a contract is an arbitrary proposition mentioning f , and one conjunct may call f several times, or under a fold. The adaptation needs an instrumented evaluation that records the argument tuples with which f 's body is entered, per conjunct, so that guards are evaluated in the right environments. For recursive programs those environments include the recursive calls, which ties this item to R3 and R5.

Closing test. Probes 9 and 10; the `max`, `min`, `filter`, `insert-sorted` and `compare` entries of the ported benchmarks.

4.2 A2. Recursion on natural numbers, literals and arithmetic

Gap. Structural recursion excludes `Nat` because nested induction on natural-number hypotheses exploded the search, and the only numeric components are `Nat.add`, `Nat.succ` and `Nat.zero`. Probe 4 (2^n) therefore has no program in the grammar.

Adaptation. Three separable changes. (i) Allow `Nat.rec` at the outermost level, and only there, exactly as for other inductives, when a contract is present; the explosion came from nested use. (ii) Treat numeric literals as components with a cost that grows with the literal, restricted to literals that occur in the contract plus 0, 1 and 2; this is the standard constant-seeding heuristic

of enumerative synthesis. (iii) Add `Nat.mul`, `Nat.sub`, `Nat.pred` and the decidable comparisons as curated components behind the demand filter. None of this is new; the adaptation is making the residual evaluator efficient on `Nat.rec` terms, where kernel reduction of literals uses the GMP acceleration only for closed numerals and falls back to unary reduction when a hole is nearby.

Closing test. Probe 4, factorial, Fibonacci with a pair accumulator (which needs R2), sum to n , and the `nat_*` entries of the Myth suite.

4.3 A3. Normal forms and symmetry reduction

Gap. The traces show the same program reached along several paths and many programs that differ only by $\beta\eta$ or by a trivial identity (`fun a x => x` as a fold step is the identity fold). Closed programs are memoized after refutation, but partial programs are not, and nothing prevents generating the redundant ones.

What is known. Proof search in focused calculi enumerates only β -normal η -long forms, which removes all $\beta\eta$ -duplicates by construction; this is the basis of inhabitation tools from Djinn onward and of the recent Canonical solver for Lean [6]. Bottom-up synthesizers prune by observational equivalence: two terms with equal values on all examples are interchangeable, so only one is kept [33, 12].

Adaptation. The engine’s rules already build η -long forms (introduction is forced and exclusive) and never build a β -redex except through forward application (rule 7b), which should be restricted to results that are then deconstructed. What is missing is a check that an *exact-local* rule is not used where an η -expanded application would produce the same term, and symmetry reduction for commutative components. Observational equivalence carries over only for closed subterms of ground type whose free variables are all bound by the observations, which is the minority in a top-down search; its proper home is the bottom-up component of Section 5.

4.4 A4. Parametricity-directed test inputs

Idea. A polymorphic function cannot inspect its type argument. For `f : (A : Type) -> List A -> List A`, the free theorem [36] says f commutes with `map`; hence f is determined by its behavior on lists of distinct elements, for instance on `[0,1,...,n-1]` at type `Nat`. Work on testing polymorphic properties turned this into a method for choosing a single monomorphic instance that is as good as testing all instances [4].

Use in Leant2. Three uses, in increasing ambition. (i) *Deduplicate* accepted candidates: two candidates of a polymorphic type are equal as functions up to a size bound if they agree on the canonical inputs, so ranking can show one representative per behavior and say how many were merged. (ii) *Explain* candidates: print each candidate’s action on the canonical input (`[0,1,2] |-> [2,1,0]`), which is more readable than the term. (iii) *Warn* about under-specified contracts: if accepted candidates disagree on a canonical input, show the input as a question to the user. The adaptation work is computing the canonical instance for Lean types with type classes, where parametricity holds only relative to the class laws, and for dependent types, where the free theorem is more delicate. For plain polymorphic first-order signatures, which is most of the corpus, the published construction applies directly.

4.5 A5. Rule indexing

Each rule scans all locals and all providers and asks the unifier whether they fit. Lean’s `DiscrTree` and `LazyDiscrTree` index terms by their rigid head structure and answer “which entries could unify with this goal” in time proportional to the goal, not the index; `exact?` and `simp` are built on them, and Aesop uses them to select rules [2]. Index providers by conclusion once per query, and index locals incrementally as they are introduced (the index is persistent, so it backtracks with the state for free). Two things do not carry over: conclusions that are a bare type variable match every goal, so the demand filter stays; and the index does not see through reducible definitions unless keys are normalized the way `simp` normalizes them, which has to match what the unifier will later accept.

Closing test. With two thousand session declarations in scope, time per node grows by less than 10% over the empty session.

Part III

Open problems

5 R1. Search control with coupled goals under a wall-clock budget

5.1 The problem

Given a query and ten seconds, in what order should partial programs be expanded? The engine answers with a fixed rule order, a fixed goal order modified by three deferral limits, depth-first search and iterative deepening on depth. Section 1.3 shows what this costs: one rule opened 90% of a search tree and nothing in the engine could know it; a depth-4 pass ran for 4.7 s without a candidate while the depth-5 pass found one in 0.5 s; and a cut pass is repeated from the start by the next lane.

Three sub-problems are tangled together.

Node order. Which open partial program to expand next. Depth-first order is forced today by the representation: the only saved states are those on the current path.

Goal order. Within a partial program, which hole to fill next. Holes share metavariables, so the order changes the branching factor of the others by orders of magnitude. The deferral rules are hand-written answers to four recurring situations (type holes, instance goals, holes of unknown type, the contract).

Budget split. How to divide wall-clock time between lanes and between depths, when the time a pass needs is not known in advance.

5.2 Why it is hard here

Coupled goals. In first-order proof search without shared variables, subgoals are independent and an AND/OR tree can be searched best-first with memoized subgoals. With shared metavariables the subgoals of one node are a constraint problem: solving one instantiates the others. Aesop meets the same difficulty in best-first proof search for Lean and handles it by copying goals that share metavariables, so that each assignment lives in its own branch, at the price of duplicated work and a more complicated tree [2]. Leant2 sidesteps the difficulty by keeping all

unfinished goals in the continuation, which makes depth-first search correct and makes any other order expensive. We do not know of a search organization that has both properties: independent treatment of goals that really are independent, and correct joint treatment of those that are not, without either copying or sequentializing.

No cost signal. Weighted enumeration needs a cost per production. Depth is a poor proxy: the accumulator rule and an exact local both cost one unit. A good cost is the expected size of the subtree a rule opens times the probability that the solution is inside, neither of which is known before the search.

Wall clock, not node count. Node cost varies by a factor of a hundred between a type-only query and a contract query with forty observations. Budgets in nodes do not transfer; budgets in time make the search non-reproducible across machines unless the cut points are made irrelevant to the result.

5.3 What is known

- Iterative deepening is asymptotically optimal among brute-force tree searches when the branching factor is large and nothing is known about costs [18]; its overhead is the factor $b/(b-1)$. It stops being a good choice exactly when passes are cut by a deadline rather than completed, which is our situation.
- For a randomized or restarted procedure with unknown run-time distribution, the universal restart schedule is within a logarithmic factor of optimal [22]. Lanes and depths are such a portfolio.
- Enumerative synthesizers replace depth by a *cost* from a probabilistic grammar and enumerate in cost order; the grammar can be learned offline (DeepCoder [9]) or during the search from partial successes (Probe’s just-in-time learning [25]).
- Choosing which hole to fill next is variable ordering in constraint satisfaction, where fail-first heuristics (smallest domain, most constrained) are standard and effective. Agda’s Agsy orders subproblems by a similar principle [3].
- Best-first search over proof states with rule priorities and a normalization phase is what Aesop implements for Lean, with white-box rule sets [2]. Canonical searches β -normal η -long terms of dependent type theory with its own term representation and heuristics for the order of metavariable refinement, and reports large gains from that order [6].

5.4 Design to try first

Step 1: make states first-class. A search state is a `Meta.SavedState`, a goal list and a cost. Saved states are cheap (Table 3: 1% of time), because the metavariable context is a persistent structure. Keep a priority queue of states instead of a call stack. Depth-first search is the special case of a LIFO queue, so nothing is lost and the baseline can be re-verified before any policy changes.

Step 2: cost instead of depth. Give every rule application a cost, initially hand-set from the measurements (exact local 0, constructor 1, application of a local 1, provider 2, case split 3, recursion 4, accumulator instantiation 5), and expand in order of cost-so-far plus a lower bound on the cost to close the remaining goals (one unit per open non-leaf goal). This is A* with an admissible but weak heuristic, and it subsumes iterative deepening without repeated passes.

Step 3: learn costs within the query. Following Probe, when a partial program survives more observations of the contract than its siblings, lower the cost of the rules that built it for the rest of the query. The residual evaluator of Section 7 provides the signal (how many conjuncts, or how many observations inside one conjunct, are already true). This is where the two problems meet.

Step 4: goal order by estimated domain size. Replace the three deferral limits by one rule: fill next the hole with the fewest applicable rule instances, computed from the index of A5, with ties broken toward holes whose assignment would instantiate the most other goal types. Instance goals and type holes then fall out as special cases rather than as separate policies.

Step 5: a bottom-up companion. For contracts with input-output observations, enumerate small closed terms over the locals of each observation environment bottom-up, keeping one term per observational-equivalence class [33, 12], and offer them to the top-down search as additional exact candidates for holes of ground type. This is how Burst supplies the components of a recursive body [5].

5.5 What would count as progress

Solved count at 1 s, 10 s and 60 s on the benchmark suite of E8; the ratio of nodes expanded to nodes on the solution path; and variance of the result under a $\pm 30\%$ change of machine speed, which has to be zero for the candidate list and small for time.

5.6 Questions for experts

1. Is there a principled search organization for goals that share metavariables that is neither Aesop's copying nor our sequentialization? In particular, can independence be detected cheaply (no shared unassigned metavariables after instantiation) and exploited by memoizing solved independent subgoals across branches?
2. What is the right admissible heuristic for the cost to close a goal in a dependent type theory, where closing one goal can close others by unification?
3. Does just-in-time cost learning stay sound for ranking, that is, can it change *which* candidates are found within the budget in a way users would find arbitrary, and how do published systems control that?
4. For wall-clock budgets, is there a known way to make results independent of machine speed other than fixing a node budget per lane and calibrating it at start-up?

Related projects and fields: Aesop studies best-first proof search in Lean and the coupling of goals through metavariables; Canonical studies the order of metavariable refinement for inhabitation in dependent type theory; Probe and EUSolver study cost-guided and divide-and-conquer enumeration; constraint programming studies variable-ordering heuristics under dynamic domains.

6 R2. Carriers, accumulators and auxiliary state

This is the problem we consider hardest and most valuable. It is the reason for all thirteen stretch failures and for probe 11, and it reappears in every recursion scheme.

6.1 The problem

A Church-encoded list is its own fold: $\mathbf{xs} : (\mathbf{R} : \mathbf{Type}) \rightarrow (\mathbf{A} \rightarrow \mathbf{R} \rightarrow \mathbf{R}) \rightarrow \mathbf{R} \rightarrow \mathbf{R}$. To use it the synthesizer must choose R . A recursor is the same with the motive in place of R . When the function to be synthesized is a fold into its result type, R is the goal type and unification finds it. Otherwise R is a type that occurs nowhere in the query:

Function	Carrier R	Why
<code>map</code> , <code>append</code> , <code>filter</code>	result type	plain fold
<code>reverse</code> , <code>foldl</code>	$T \rightarrow T$	left-to-right accumulation through a right fold
<code>foldl1</code> , <code>reduce</code> with default	Option $A \rightarrow A$, or Option A	the first element seeds the accumulator
<code>maximum</code>	Option A	same, with a comparison
<code>minMax</code>	Option $(A \times A)$	two running values
<code>at</code> (index)	$\mathbf{Int} \rightarrow A$	the index is consumed while folding
<code>fib</code>	$\mathbf{N} \times \mathbf{N}$	tupling
tree flatten, efficiently	List $A \rightarrow$ List A	accumulator avoids quadratic append

The engine solves the first two rows. The second row needed a dedicated rule (7a: instantiate a recursive eliminator at $T \rightarrow T$ for goal T), found by hand after reading a failing trace. A general frontier of candidate carriers was tried first and multiplied the search by an order of magnitude; restricting the rule to eliminators whose step threads the result type made one probe ten times faster. That history is the problem in miniature: each carrier shape is easy once named, and enumerating shapes blindly is unaffordable.

The same problem has three other faces. *Motive generalization*: a structurally recursive function with an accumulating parameter is a recursor whose motive is a function type, motive := $\lambda_ . S \rightarrow T$. *Strengthening*: a recursive function that returns more than is asked (a pair) and is projected afterwards. *Helper invention*: a fold whose step calls an auxiliary function that must itself be synthesized. In all four the synthesizer must invent an object whose specification is not given.

6.2 What is known

Theory of folds. A classical result characterizes exactly when a function h on lists is a fold: when the kernel of h is closed under `cons`, that is, $h\ xs = h\ ys$ implies $h\ (a::xs) = h\ (a::ys)$ [37]. When it is not, h becomes a fold after pairing with auxiliary functions: this is *tupling* [35], and the third homomorphism theorem gives conditions under which a function that is both a left and a right fold is a homomorphism [32]. The smallest sufficient auxiliary information is, in automata terms, the Myhill–Nerode quotient of input lists under “indistinguishable by h after any further `cons`”. These results say a carrier *exists* and what it must distinguish. They do not give a procedure to find a *type and step function* from a handful of examples.

Synthesis systems. λ^2 fixes a set of combinators (`map`, `filter`, `foldl`, `foldr`, `recl`) and deduces examples for the combinator’s function argument from examples for the whole; for folds the deduction is possible only when the examples are closed under taking tails, and the accumulator type is the result type [19]. Myth, Smyth, Burst and Trio synthesize explicit recursion and avoid the carrier by not using folds; they need a function’s own examples to be trace-complete (Myth), or replace recursive calls by angelic values and check them afterwards (Burst), or derive recursion from straight-line blocks (Trio) [24, 28, 5, 34]. None of them invents

an accumulating parameter or a tuple unless the signature already has it. Synquid requires the user to write the auxiliary function’s refinement type [30]. DreamCoder learns reusable abstractions across tasks, which is helper invention over a corpus rather than within one query [10]. Inductive-theorem-proving research calls the proof-side twin of this problem generalization and lemma speculation and regards it as open.

So, to our knowledge: the existence theory is classical, the enumeration of carriers is not addressed by synthesis systems, and the learning of a carrier from examples has no published algorithm for this setting.

6.3 Design to try first

A carrier grammar with costs. For a goal type T , element type A and the types S_i of other locals:

$$K ::= T \mid S_i \rightarrow K \mid K \times K \mid \text{Option } K \mid A \mid \text{Bool} \mid \mathbb{N}$$

bounded by size 3, enumerated in cost order under the search of Section 5. Rule 7a is the production $T \rightarrow T$. Each carrier adds a *finishing* obligation $K \rightarrow T$ (apply to the seed, project, or supply a default for **none**), which is ordinary synthesis. The grammar is small; the cost is the subtree each carrier opens, which is why this design depends on the two ideas below and on R3.

Refute a carrier before searching under it. For a fixed carrier K , the unknowns are **step**, **init** and **finish**. The contract’s observations give, for each input list, the required value of **finish** (`foldr step init xs`). Two inputs that must produce different final results after the same suffix of **cons** operations must be mapped to different values of K . If the number of classes that the observations force exceeds the number of values of K that can be built from the inputs at the search depth, K is refuted without synthesizing anything. For finite carriers such as **Bool** this is a counting argument; for T itself it is exactly the fold-closure test of that characterization applied to the examples and explains in advance why **reverse** cannot be a plain fold into **List A** by a step of depth one. We believe this test is new as a pruning rule; whether it is strong enough to matter is an empirical question.

Learn the carrier as an automaton. Treat the observations as samples of a function on strings over A . Build the observation table used in active automata learning: rows are input prefixes (or suffixes, for right folds), columns are continuations, entries are required results. The distinct rows are the states a carrier must distinguish. The number of states and their structure suggest the carrier: one state per value of T means a plain fold; states indexed by “seen nothing” versus a value of A means **Option A**; pairs of values mean a product. This gives a *proposal* for K derived from the data, to be confirmed by synthesis and the kernel. We have not found this in the synthesis literature and do not know how it behaves with the five to forty observations that contracts provide, or with polymorphic element types, where parametricity (A4) should reduce the alphabet to positions.

Sketches as the escape hatch. Until invention works, a user who knows the carrier can write `xs (Option A) ?step ?init` as a sketch (E2). The stretch cases become a benchmark with two tracks: carrier given, and carrier invented.

6.4 What would count as progress

The thirteen stretch cases and the accumulator entries of the ported suites (`fib`, `list_rev_tailcall`, `tree_flatten`, `maximum`, `index`), solved within 10 s without new

hand-written carrier rules, and without slowing the solved Church probes by more than a factor of two.

6.5 Questions for experts

1. Is there a constructive version of the fold characterization that, given finitely many input–output pairs, returns a minimal carrier *as a type* from a fixed grammar, or proves none exists in the grammar?
2. Is the automaton view sound for polymorphic functions, and what replaces the alphabet when elements are abstract? We expect positions and equalities between positions, by parametricity, but have no proof.
3. In program calculation, tupling and accumulation are derived by fusion from a specification. Can fusion be run “backwards” from examples to propose the auxiliary function, and is there a complete strategy for a useful class (list homomorphisms, catamorphisms over polynomial functors)?
4. For recursors, is there a known enumeration of motives that is complete for primitive recursion with parameters and avoids the explosion we saw on `Nat`?
5. How do inductive theorem provers choose generalizations today, and do their heuristics (rippling, lemma speculation from failed proofs) transfer to choosing carriers from failed syntheses?

Related projects and fields: the algebra of programming and program calculation study folds, tupling and the third homomorphism theorem; λ^2 , Burst and Trio study synthesis of recursive list and tree programs from examples; automata learning studies the inference of minimal state spaces from observations; inductive theorem proving studies generalization, rippling and theory exploration.

7 R3. Evaluation of programs with holes

7.1 The problem

A contract is useful during the search only if it can reject a partial program. The engine substitutes the partial program into each conjunct of the contract and asks the kernel for the weak head normal form of `decide (C p)`. The kernel reduces until it needs the value of a hole and stops. If the result is `false` the branch is dead; if it is stuck the search goes on. This costs 45% of the time on contract queries and is the only source of pruning power the engine has. Four things are wrong with it.

1. **Nothing is reused.** Every evaluation starts from the definition of the contract. A fold over forty inputs is re-reduced from the first input at every node, although only one hole changed since the parent.
2. **All or nothing.** The evaluator reports refuted or stuck. It does not say which observations already hold, which the cost learning of R1 and the guard abduction of A1 both need, nor which hole it was stuck on in a form the search can use to choose the next goal.
3. **No backward information.** Evaluation runs forward from inputs. The literature’s strongest pruning runs backward: from the required output of the whole, derive required outputs of the holes.
4. **Recursion blocks it.** In a partial recursive body the recursive call is a call to the partial program itself and is stuck immediately, so nothing is learned until the body is complete.

7.2 What is known

Holes as values. Hazelnut Live gives an operational semantics in which evaluation proceeds around holes and produces results containing *hole closures*: a hole together with the environment in which it was reached [15]. Smyth builds synthesis on it: *live bidirectional evaluation* evaluates a sketch to a result with hole closures, then *unevaluates* the required output against that result to obtain example constraints on each hole, in the hole’s own environment [28]. This is exactly items 2 and 3 above, for a simply typed ML core with algebraic data types and first-order examples.

Angelic recursion. Burst evaluates a partial recursive program by letting recursive calls return whatever value makes the examples pass, records those assumptions, and later checks or repairs them; this removes the need for trace-complete examples at the price of a refinement loop whose termination argument is specific to their bottom-up setting [5].

Example propagation. λ^2 and Myth push examples through combinators and constructors by inversion rules written per combinator [19, 24]; a type-theoretic reading of example-directed synthesis treats examples as refinement types, so that propagation is type checking [14].

Incremental normalization. Normalization by evaluation with explicit closures makes sharing natural; memoizing it across edits of a term is, as far as we know, studied for incremental type checking in editors rather than for search.

7.3 Why it is hard here

The evaluator of record is the Lean kernel. A refutation must agree with what the kernel will later say about the closed program, or pruning loses solutions. Using the kernel itself guarantees that. A custom evaluator for speed and for hole closures has to be proved, or at least tested exhaustively, to be conservative: it may report “stuck” where the kernel would decide, never the reverse. Lean’s reduction includes definitional unfolding with reducibility hints, structure eta, recursor reduction on literals, quotient and projection rules, and proof irrelevance; a faithful reimplementaion is a large surface.

Observations are propositions, not pairs. Unevaluation in Smyth starts from an example $f\ v = w$. A Lean contract is any proposition: a conjunction of equations is typical, but so are `check f = true` where `check` folds over forty inputs and compares with `==`, and quantified statements. Backward propagation has to go through the contract’s own structure (`List.all`, `&&`, `BEq` instances, `decide`) before it reaches an equation on a value of f . Each of those needs an inversion rule or a generic treatment.

Higher-order and Church-encoded inputs. When the inputs are functions (every Church probe), the “required output” of a hole is a function, and examples on it arise only through the applications the rest of the program performs. Smyth’s unevaluation is first-order.

Dependent types. A hole’s environment may bind types and proofs; two hole closures for the same hole can have different types. Example constraints must be typed per closure.

7.4 Design to try first

Stage 1: a richer report from the kernel path, no new evaluator. Decompose the contract further than its top-level conjuncts: recognize `List.all xs p = true`, `(a && b) = true` and `decide P = true` and split them into independent observations at query time, each with its own precomputed decider. The evaluator then reports a bit vector (true, false, stuck) per observation. This gives R1 its signal and A1 its satisfied sets, keeps the kernel as evaluator, and makes each evaluation smaller. The “stuck on these holes” memo that exists today becomes per observation, so an assignment re-evaluates only the observations blocked on that hole.

Stage 2: a conservative hole-closure evaluator for a fragment. Implement call-by-need evaluation with environments for the fragment the engine actually generates (lambda, application, constructors, recursors with constant motive, projections, literals with the arithmetic the kernel accelerates), over Lean `Expr` with holes as neutral terms that record their environment. Fall back to the kernel outside the fragment. Validate by differential testing against `Kernel.whnf` on every closed program the harnesses generate (about 10^5 per full run), requiring agreement on decided cases.

Stage 3: unevaluation for first-order observations. For observations of the form $f \vec{v} = w$ with w of a first-order data type, push w backward through constructors and case analysis to hole closures, as in Smyth, and use the resulting examples in two ways: to refute (no term at this depth can produce that value from this environment) and to order goals (fill the hole with the most constrained examples first, which is Step 4 of Section 5).

Stage 4: angelic recursive calls. In the body of a structural recursion the induction hypothesis is a local variable. Evaluate with it bound to an oracle that answers from the contract’s observations when the argument is one of the observed inputs (or a sub-term of one), and is stuck otherwise. For trace-complete contracts this is Myth’s method; for others it degrades to today’s behavior rather than to unsoundness, since a stuck call refutes nothing.

7.5 Soundness conditions

Invariant 7.1 (Conservative refutation). If the evaluator reports an observation `false` for a partial program p , then for every completion p' of p the kernel decides that observation `false` for p' .

For the kernel path this holds because reduction never inspected a hole. For Stage 2 it must be established for the fragment. For Stage 4 it holds only if the oracle’s answers are consequences of the contract: an oracle answer taken from an observation $f v = w$ is the value every accepted f must return at v , so a branch refuted under it is refuted for every program that satisfies the contract, which is the set we care about. This argument fails for contracts that do not determine $f v$ uniquely, and the implementation has to check that the observation it uses is an equation with a closed right-hand side.

7.6 Questions for experts

1. Has live bidirectional evaluation been extended to higher-order examples or to dependent types? If not, where does it break first?
2. Is there a published incremental or memoized normalizer whose invalidation is driven by hole assignment, suitable for a backtracking search in which assignments are undone?

3. What is the cheapest sound way to obtain a fast evaluator that provably under-approximates the Lean kernel’s conversion on a fragment? Would extracting the fragment’s rules from the kernel’s specification in Lean4Lean be realistic?
4. For angelic recursion in a top-down search, is there a termination and completeness argument analogous to Burst’s?

Related projects and fields: Hazel studies hole closures and live evaluation of incomplete programs; Smyth studies bidirectional evaluation for sketches; Burst studies angelic execution of recursive calls; Lean4Lean specifies the Lean kernel’s reduction rules in Lean; normalization by evaluation studies efficient normalizers with neutral terms.

8 R4. Indexed families and the motive problem

8.1 The problem

Leant2’s reason to exist is Lean’s type theory, and this is where it is weakest relative to its ambition. Probe 7 (head of a non-empty vector) works, because Lean’s `cases` performs dependent elimination and unifies indices. Probe 8 does not:

```
(A B : Type) -> (n : Nat) -> (A -> B) -> Vec A n -> Vec B n
```

The solution is an induction on the vector whose motive depends on the index, `fun n _ => Vec B n`. The engine’s recursion rule calls `induction` with whatever motive Lean’s tactic infers after the context has been fixed, never generalizes other hypotheses, and treats indexed families as out of scope. Four related capabilities are missing.

Motive inference. Choosing the motive of a recursor application so that the branches type check and the result has the goal type. For non-indexed families with a constant motive this is trivial. In general it is higher-order unification: find M with $M \vec{i} x \equiv G$.

Generalization. Deciding which hypotheses to revert before induction. Reverting too little makes the induction hypothesis useless (it speaks about the wrong index or the wrong accumulator); reverting everything makes every branch a function and multiplies the search. This is the same problem as the carrier of Section 6.

Transport. Goals of type `Vec A (n + 0)` when a term of type `Vec A n` is at hand. The term exists only after rewriting with a propositional equality, and the rewriting itself needs a motive.

Type-level computation. Goals whose type is a function applied to a hole, `F ?n`, where progress needs unfolding `F` on a guessed constructor form of the hole.

8.2 What is known

Higher-order unification is undecidable in general [17]; the pattern fragment is decidable with most general unifiers [23], and practical elaborators extend it with postponement and pruning [1]. Lean’s elaborator solves the motive problem for `induction`, `cases`, `match` and the triangle rewrite notation by abstracting occurrences of the indices and the major premise in the goal, which is a heuristic: the “motive is not type correct” error is its failure mode and is a familiar obstacle for Lean users. Dependent pattern matching by unification of indices is well understood [8].

For *search* in dependent type theories there are several tools and no consensus. Agsy searches Agda terms with case splitting and a notion of refinement order [3]; Mimer is its current successor in Agda. `sauto` searches for inhabitants in the Calculus of Inductive Constructions with a focused, eager treatment of inversion and explicit limits on dependent elimination [26]. Canonical searches for β -normal, η -long inhabitants of a dependent type theory with its own representation and

reports solving goals by structural recursion through recursors when the motive is determined by unification [6]. Idris has type-directed proof search with case splitting in the editor. All of them bound or avoid exactly the four capabilities above: none, to our knowledge, searches over motives or generalizations, and none handles transport other than by the user’s rewriting.

8.3 Why it is hard

The space of motives is a function space over types. Even restricted to abstraction of sub-terms of the goal, a goal with k occurrences of index expressions has 2^k candidate abstractions, most ill typed. Generalization multiplies by the subsets of the context. Transport makes the term language non-canonical: there are infinitely many proofs of an index equation and the kernel term contains them, so size-based enumeration and syntactic deduplication stop making sense, while proof irrelevance says all these terms are the same program. And every step relies on definitional unfolding, where the choice of transparency decides both whether unification succeeds and how long it takes; Lean automation in general has no principled answer to when to unfold.

8.4 Design to try first

1. **Index-generalizing induction.** For an inductive family local $x : I$ ps is, revert x , the index variables among is, and every hypothesis whose type mentions them; then apply the recursor with the motive Lean infers for the reverted goal. This is the **induction ... generalizing** discipline, applied automatically and only to index variables, which keeps the branching at one motive per local. Probe 8 becomes solvable: the branches are $\text{Vec } B \ 0$ and $B \rightarrow \text{Vec } B \ n \rightarrow \text{Vec } B \ (n+1)$ with the right hypothesis.
2. **Motives from the carrier grammar.** For accumulating recursion, the generalized hypotheses are chosen by the carrier search of Section 6: a carrier $S \rightarrow T$ is “revert the local of type S ”. One mechanism then serves Church folds and recursors.
3. **Transport as a proof obligation, not a term.** When unification fails only because two index expressions are not definitionally equal, create the goal $i = j$, send it to the arithmetic tier of the portfolio (`omega`, `simp_arith`, `grind`), and insert a cast on success. Count the cast as zero size for ranking and erase it for display. Bound the number of casts per candidate to two.
4. **Constructor-form splitting of index holes.** When a goal type is stuck on a hole of an inductive type, split the hole into its constructor forms (narrowing), bounded by the split budget.
5. **A differential benchmark against existing tools.** Run Canonical and `exact?` (and, outside Lean, `sauto` and Agda’s `auto` on translated problems) on the dependent part of the benchmark suite. The point is to learn where each stops, not to compete.

8.5 Questions for experts

1. Is there a complete and terminating enumeration of motives for a useful class, for instance motives that abstract a subset of the *maximal* occurrences of index terms, and is that class closed under the generalizations needed for accumulators?
2. How should a search treat propositional transports so that enumeration stays canonical? Is working in a setoid or with heterogeneous equality internally, and producing casts only at the end, known to work?
3. What is the experience of Canonical and `sauto` with definitional unfolding during search: fixed transparency, on-demand unfolding when stuck, or normalization up front?
4. For Lean specifically: can the elaborator’s motive computation be called as a library with a failure that is cheap and informative, so that a search can try several abstractions?

Related projects and fields: the Lean elaborator implements motive computation for **induction**, **match** and rewriting, and transparency control; Canonical and **sauto** study inhabitation search in dependent type theories; Agsy and Mimer study term search with case splitting in Agda; research on dependent pattern matching and higher-order unification studies index unification and the pattern fragment.

9 R5. General recursion from incomplete examples

9.1 The problem

The engine has one recursion scheme: the recursor of an index-free recursive inductive type other than **Nat**, with a constant motive, at the outermost level, with the induction hypotheses as the only recursive calls. It solves tree size, list length, list map and the Leant recursive scenarios. It cannot express, and so cannot find:

- recursion on **Nat** (item A2), on two arguments at once (**zip**, **merge**), or on a derived measure (**gcd**, division, binary search);
- recursive calls on sub-sub-terms (**Fibonacci**, **pairs**), which need course-of-values recursion;
- accumulating parameters and tupled results (Section 6);
- mutual and nested inductive types, for which Lean generates recursors that are awkward to use directly;
- helper functions: probe 11 (in-order flatten) is `flatten l ++ [a] ++ flatten r`; with **append** available as a provider the difficulty is depth, since the body has two recursive results combined by two applications inside a constructor context, and the search tree at that depth is large under a fixed rule order.

9.2 What is known

Schemes and termination. Lean accepts recursive definitions by structural recursion (compiled to **brecOn**, which is course-of-values) and by well-founded recursion with a termination measure; recent Lean versions guess lexicographic measures automatically for many definitions. Termination analysis by size-change [27] and by lexicographic orders is mature. So *checking* a candidate recursive definition is solved by handing it to Lean’s definition elaborator; the cost is tens of milliseconds per candidate, far above a kernel check.

Synthesis. Myth synthesizes recursive functions by treating the function’s own examples as its specification for recursive calls, which requires trace-complete examples [24]. Escher, λ^2 and Smyth have variants of the same requirement [12, 19, 28]. Burst removes it with angelic execution and a refinement loop [5]; Trio synthesizes straight-line “blocks” from examples and then folds them into a recursive definition [34]. Synquid proves termination through a user-supplied measure in the refinement type [30]. None targets a language where the definition must also pass a kernel as a term built from recursors.

9.3 Why it is hard here

Two representations. The object the kernel certifies is a recursor term (or a **WellFounded.fix** term with a proof). The object a search wants to enumerate is a recursive equation with calls to a named function. Translating every candidate through Lean’s definition elaborator is too slow for the inner loop; enumerating recursor terms directly makes course-of-values and multi-argument recursion unnatural.

Contracts are rarely trace-complete. A user who writes $f [1,2,3] = 6$ seldom adds $f [2,3] = 5$ and $f [3] = 3$. Without them a partial recursive body cannot be evaluated (R3), so recursion is searched blind until the body closes. The engine survives today because the bodies are tiny.

The scheme is a carrier. Choosing the argument to recurse on, the measure, the accumulators and the tupling is the carrier problem again, with termination as an extra constraint.

9.4 Design to try first

1. **A capability-based body language.** Keep the unified proposal’s principle: recursion is admitted only through typed *recursive-call capabilities*. Generalize the single capability “the induction hypothesis of this constructor argument” to a capability $rec : (y : \tau) \rightarrow y \prec x \rightarrow \sigma$, a function that may be called on any argument provably smaller than the current one in a chosen order $\prec$. For structural orders the proof is a constructor path found syntactically; for measures it is an arithmetic goal for **omega**. The search enumerates bodies against this capability; the *realization* step, once per closed candidate that passes the contract by evaluation, compiles the body to **brecOn** or **WellFounded.fix** through Lean’s definition elaborator and kernel-checks it. Evaluation during search uses a direct interpreter for the capability (unfold the body on smaller arguments, with fuel), which is sound for refutation because the realized term is definitionally or provably equal to that unfolding.
2. **A small, ordered scheme grammar.** Structural on one argument; structural on one argument with one accumulator (from the carrier grammar); simultaneous structural on two arguments; course-of-values of depth two; measure **sizeOf** or length of one argument. Enumerate in that order under the cost search of R1.
3. **Trace completion by the evaluator.** Stage 4 of Section 7: recursive calls on observed inputs are answered from the contract. Add Burst’s idea in its simplest form: when a call is on an unobserved input, record the value that would make the observation true as an *assumption* and continue; a closed candidate is accepted only after all assumptions are discharged by actually evaluating the candidate, which the closed-program check does anyway. Assumptions then only *order* the search; they never prune, so soundness does not depend on them.
4. **Helpers from failed searches.** When a body search fails at depth d with a hole whose examples (from unevaluation) are consistent, promote the hole to a separate synthesis problem with those examples as its contract. This is how probe 11 would find **append** if it were not a provider. It is bounded by allowing one helper per query.

9.5 What would count as progress

The recursion entries of the ported suites at 10 s, reported separately for trace-complete and trace-incomplete contracts; probe 4 and probe 11; no change in the scores of Table 1.

9.6 Questions for experts

1. Is the search-time interpreter of a capability-based body guaranteed to agree with the realized **brecOn** or **WellFounded.fix** term up to the kernel’s conversion, or only propositionally? For well-founded recursion Lean’s terms do not reduce well in the kernel; what is the recommended way to evaluate them (equation lemmas, **decide** with **Nat.rec** fuel, compiled evaluation under a trusted-code profile)?
2. Does angelic execution compose with top-down type-directed search, and what is lost relative to Burst’s bottom-up setting?

3. Is there a canonical small basis of recursion schemes that covers the functions in standard introductory corpora, with measured coverage?

Related projects and fields: Lean’s definition elaborator implements structural and well-founded recursion and the guessing of termination measures; Burst, Trio, Myth and Synquid study synthesis of recursive programs from examples and refinement types; termination analysis studies size-change and lexicographic orders.

10 R6. Contracts beyond decidable observations

10.1 The problem

Almost every contract in the corpus is a conjunction of closed decidable equations. Those are decided by evaluation and the proof is `of_decide_eq_true`. The interesting contracts are different:

Universal properties. `forall xs, (f xs).length = xs.length; forall xs, f (f xs) = xs; forall xs, Sorted (f xs) and Perm xs (f xs).`

Refinement-style signatures. `(xs : List A) -> Subtype (fun ys => ys.length = xs.length)`, where the contract is in the type and the program must carry proofs.

Relational and metamorphic properties. `f (xs ++ ys) = f xs ++ f ys;` agreement with a reference implementation on all inputs.

For these the engine has a five-tactic portfolio applied to the closed program. It proves the identity function length-preserving. It cannot prove that a synthesized recursive `map` is, because that proof is an induction whose statement mentions the just-synthesized recursor term, and no tactic in the portfolio performs induction.

Two problems follow. *Refutation*: wrong candidates must be rejected quickly, without a proof attempt. *Certification*: the right candidate must be proved, automatically, within the budget.

10.2 What is known

Refutation. Counterexample-guided inductive synthesis alternates a synthesizer that satisfies finitely many examples with a verifier that either proves the candidate or returns a new counterexample [7]. In Lean the verifier’s cheap half is testing: `decide` on bounded domains, and property-based testing with generators and shrinking (the Plausible library). A counterexample is a closed instance of the contract and is kernel-checkable, which is the “tests refute, proofs certify” rule of the unified proposal.

Certification. Synquid checks refinement types with an SMT solver over decidable theories and relies on measures to make recursive functions’ properties inductive by construction [30]. Leon and related tools verify synthesized recursive functions by unfolding plus SMT. In Lean the available automation is `simp`, `omega`, `grind`, `decide`, `Aesop`, `hammers` that call external provers with premise selection [21], and neural provers. Automated induction (choosing the induction variable, generalizing, finding lemmas) remains a research area with specialised provers and no robust general solution.

Correct by construction. The alternative to proving after the fact is to synthesize program and proof together: the target is the subtype, the search builds a dependent pair, and the proof obligations of each branch are local (in the `cons` branch, prove the property from the induction hypothesis’s property). This is how type-theoretic program extraction has always worked; as a *search* strategy it makes every branch carry a proof goal with holes in it.

10.3 Why it is hard

Proof search for a property of a program with holes is not meaningful until the holes that the property depends on are filled; but waiting until the program is closed gives the prover a monolithic goal about a recursor term, the form automation handles worst. The local obligations of the correct-by-construction route are easy (often `simp` with the hypothesis) but there are many of them, each tactic call costs one to a hundred milliseconds, and the search makes thousands of nodes. The schedule that pays proof cost only on promising branches (“proof debt” in the unified proposal) has no known optimal policy; it is a multi-armed bandit with correlated arms.

10.4 Design to try first

1. **Bounded instantiation as a first-class refuter** (E7), extended with generators: for `forall x : tau, P x`, enumerate small values of `tau` by the engine’s own type-directed enumeration (it is a synthesizer of inhabitants, after all), most-canonical first by A4. Cache the counterexamples found for one candidate and test them first on the next: this is CEGIS with the engine as its own example generator.
2. **Program and proof together for subtype targets.** When the target is a subtype or the contract is a universally quantified property of the result, run the recursion rule on the *pair*: the motive is the subtype, each branch must produce a value and a proof, and the proof goal is deferred (it is a proposition about open holes, so the existing rotation rule already sends it last) until the branch’s value is closed. Then it goes to the portfolio with the induction hypothesis’s property in context. The proof obligations are local and first-order in the typical cases (length, membership, ordering with `omega`).
3. **Proof debt with a fixed exchange rate.** Charge proof attempts to the wall clock like any other step, but order them: never attempt a proof for a candidate that has not survived bounded instantiation; attempt local obligations before global ones; give each obligation a tier ladder (5 ms `rfl/decide`, 50 ms `simp/omega`, 500 ms `grind`) and climb it only when the rest of the search is idle or exhausted.
4. **Inductive proofs of closed recursor terms** as the fallback: a small tactic that inducts on the recursor’s major premise, unfolds the recursor’s computation rule, and calls the portfolio with the hypothesis. It covers the properties whose proof follows the program’s recursion, which is most of those a user states.
5. **Report unproved survivors honestly.** A candidate that passes all tests and no proof is a distinct outcome (“unrefuted, not certified”) and must never be shown as verified; the unified proposal’s result algebra already has the slot.

10.5 Questions for experts

1. For the program-and-proof search, is there a known discipline for interleaving value holes and proof holes that avoids both premature proof attempts and late discovery that a value choice is unprovable? Refinement-type systems use liquid abstraction for this; is there an analogue without an SMT solver and with Lean’s propositions?
2. Which class of properties of structurally recursive programs is provable by “induct along the program, simplify, close with arithmetic”, and can membership in the class be recognized syntactically so that the engine does not waste the budget outside it?
3. What is the state of automated induction in Lean today (rippling-style tactics, lemma discovery, `grind` with induction, hammers), and which of it is usable as a library with a time limit?
4. Is there a principled scheduling policy for proof debt, for example from the literature on anytime algorithms or bandits with switching costs?

Related projects and fields: Synquid and refinement-type verifiers study synthesis and checking against refinement types; `grind`, Aesop and the Lean hammer are the general-purpose proof automation available in Lean; inductive theorem proving studies automated induction and lemma discovery; property-based testing studies generators for structured and dependent data.

11 R7. Component retrieval at library scale

11.1 The problem

The engine knows twenty curated library functions and whatever the session declares. Probes 14 and 16 fail because `Nat.succ_pos` and `Nat.le_trans` are not among them. Lean core has tens of thousands of declarations; Mathlib has several hundred thousand. A synthesizer for “Lean types that occur in practice” has to find, among them, the handful that belong in the program, inside a search whose branching factor is already the limiting resource.

The demand filter shows the shape of the difficulty. A provider whose conclusion is a constant-headed type is tried only on goals with that head, which is cheap and precise. A provider whose conclusion is a type variable, such as `List.foldr`, `Option.getD` or `id`, fits every goal; the engine admits it only when a local’s type head occurs among its argument heads. That rule was enough for twenty functions. It will not be for twenty thousand.

11.2 What is known

Lookup in Lean. `exact?` and `apply?` index the library’s conclusions in a lazily built discrimination tree and try the matches, closing side goals with `solve_by_elim`. They find single lemma applications, not compositions. Loogle answers pattern queries over the same data. Premise selection for hammers ranks declarations by relevance to a goal with symbol-overlap heuristics or learned embeddings [21, 20].

Component-based synthesis. SyPet models an API as a Petri net whose places are types and whose transitions are methods, and finds compositions as reachability paths, then fills arguments by SAT [31]. Hoogle+ extends this to polymorphic Haskell by *type-guided abstraction refinement*: search over an abstraction of the types in which polymorphic components are approximated, check the concrete types of a found path, and refine the abstraction where the check fails [16]. Both target tens of thousands of components and first-order or rank-1 types, without dependent types, universe levels or implicit instance arguments.

11.3 Why it is hard here

- **Dependent and polymorphic conclusions.** A Lean lemma’s conclusion mentions its arguments; “types as places” has no direct meaning when types contain terms. The abstraction must forget term indices in a way that is refinable.
- **Type classes.** Most library functions are generic over a structure (`[Add A]`, `[LE A]`, `[Monad m]`). A component is applicable only if instances resolve, and instance resolution is itself a search that is too slow to run per candidate per node.
- **Programs versus proofs.** Relevance signals learned on proof corpora rank lemmas. For data goals the relevant components are functions, and which function is relevant is determined by the contract’s behavior rather than by symbols in the goal.
- **Cost of loading.** Importing Mathlib takes seconds and gigabytes before the first node is expanded. An index has to be built once and stored, keyed by the environment’s fingerprint.

11.4 Design to try first

1. **Two retrieval modes.** For proposition-typed holes, call the existing lookup (`exact?`'s index, then the premise selector) with a time limit and take the top k as providers for that hole only. This alone fixes probes 14 and 16 and costs nothing when no proposition-typed hole remains open after the ordinary rules.
2. **A reachability pre-pass for data goals.** Build, once per environment, a graph whose nodes are type heads with arity (the abstraction: `List _`, `Option _`, `Nat`, a node for "type variable") and whose edges are functions from argument heads to result head. Given a query, compute the components that lie on some path from the heads available in the context to the head of the goal, within length three. Only those become providers. This is the first level of Hoogle+'s abstraction, without refinement; refinement is what the ordinary search does when it unifies.
3. **Behavioral relevance.** When the contract has input-output observations, run each unary or binary candidate component of fitting type on the observed inputs. A component whose output equals the required output on all observations is the answer; one whose output is a prefix, a permutation or a sub-term of the required output gets a lower cost. This is a generic form of what λ^2 does by hand-written deduction rules and is cheap because evaluation of closed library functions can use compiled code under a trusted-evaluation profile, with the kernel confirming only the winner.
4. **Instance-aware indexing.** Index class-generic components under the class as a precondition, and resolve instances once per (class, type) pair per query, memoized.

11.5 Questions for experts

1. What is the right type abstraction for dependent, universe-polymorphic signatures so that abstract reachability is a useful over-approximation? Erasing all term arguments is sound but may connect everything to everything.
2. Has type-guided abstraction refinement been tried with type classes, and how were instance constraints represented in the net?
3. Can Lean's premise-selection interface be queried with a *data* goal plus examples, and is there a corpus from which relevance for programs (rather than proofs) could be learned, for instance definitions in Mathlib and Std with their bodies erased?

Related projects and fields: Hoogle+ and SyPet study type-directed component-based synthesis over large libraries; `exact?`, Loogle and Lean's premise-selection interface implement lookup and relevance ranking over Lean libraries; LeanDojo and the Lean hammer study learned premise selection.

12 R8. Ranking and candidate equivalence

12.1 The problem

A type usually has many inhabitants and a loose contract admits several behaviors. The engine returns up to twelve candidates sorted by three keys: number of unused explicit inputs, number of eliminators, term size; duplicates are removed by printed form. This produced sensible first candidates on the corpus after two corrections found by reading output (instance binders are not inputs; class instances are projected, not destructured). It is a heuristic with no definition of what it approximates. Concretely:

- For queries whose answer must invent a type former the list is twelve variations of one idea (`fun _ => Empty`, `fun _ => Unit`, ...); eleven slots carry no information.

- When the contract under-determines the function, two behaviorally different candidates appear as siblings with nothing to say that they differ, or where.
- The grace period ends the search 400 ms after the first candidate. Whether the best candidate has been seen by then is not known.

12.2 What is known

Synthesizers rank by size or by a probabilistic grammar learned from code [9, 25]; programming-by-example systems such as FlashFill rank with learned features because users give one or two examples and the intended program is rarely the smallest. Type-directed tools (Djinn, Agsy, `exact?`) return the first or the smallest. Parametricity gives exact equivalence tests for polymorphic first-order functions (A4). For general Lean terms, equivalence is undecidable, and definitional equality is both too fine (it separates extensionally equal functions) and occasionally expensive.

We know of no published notion of *what the right first answer is* for type-and-contract queries that could be optimized, nor of a study of what users of a proof assistant expect from such a tool.

12.3 Design to try first

1. **Behavioral classes.** Group candidates by their values on a probe set: the contract’s own inputs, the canonical parametric inputs of A4, and small values enumerated as in R6. Show one representative per class (the best by the present keys) with the count of merged candidates. Two classes that the contract does not separate are shown with a distinguishing input.
2. **Relevance as a first-class key.** A candidate that ignores an input, returns a constant, or discards a recursive result is suspect in a way that is independent of size. Generalize “unused inputs” to *information flow*: which inputs can influence the output, computed syntactically on the normal form. Rank by flow first.
3. **An anytime stopping rule.** Replace the fixed grace period by: stop when the cost frontier of the search (R1) exceeds the cost of the best candidate by a margin, since no cheaper candidate can then appear. With a cost-ordered search this is exact for the cost model; the grace period remains as an upper bound.
4. **A ranking benchmark.** For each benchmark problem record the reference solution and report its rank. This is the only way to tell whether a change to ranking is an improvement.

12.4 Questions for experts

1. Is there a defensible formal objective for ranking inhabitants of a type, for example minimum description length under a grammar learned from Lean’s library, and does it agree with user preference in studies from programming-by-example?
2. For dependent types, which notion of candidate equivalence is both cheap and meaningful: definitional equality after normalization with proof irrelevance, or observational equality on generated inputs?

Related projects and fields: programming by example studies ranking of consistent programs and user expectations; research on parametricity studies exact testing of polymorphic functions; human-factors research studies how proof-assistant users work with automation.

13 R9. Learned guidance under a kernel gate

13.1 The problem

Every open problem above contains a guess that a statistical model might make better than an enumeration: the carrier type, the generalization, the helper’s specification, the relevant library components, the rule order, the guard. Large language models write plausible Lean and are often right about exactly these creative steps, while being unreliable about details that a type checker and a contract catch immediately. The kernel gate makes the combination safe by construction: a model can only propose. The open questions are whether the combination is worth its cost inside a ten-second interactive budget, and how it coexists with the owner’s constraints of determinism, no settings, and a core-only default.

13.2 What is known

Neural guidance of enumerative synthesis (DeepCoder [9] and successors) learns a distribution over components from the examples and reorders the search; library learning (DreamCoder [10]) grows the component set across tasks. For Lean, retrieval-augmented provers and hammers [20, 21] show that premise selection transfers; whole-proof and step-level neural provers have improved quickly. We do not know of work on neural guidance for *program* synthesis in a dependently typed language with a contract, nor of measurements of latency-bounded use.

13.3 Design to try first

1. **One interface, several sources.** Define a proposal interface: given the query, the partial program and a named decision point (carrier, generalization, helper type, component set, guard), return an ordered list of well-formed Lean terms or types. The enumerative grammars of R2 and R7 are the default implementation. A model is another implementation. Everything downstream (type checking, residual evaluation, the gate) is unchanged.
2. **Proposals only reorder.** A learned source may change the order in which alternatives are tried, never the set, so completeness up to the bound and the meaning of a negative result are preserved.
3. **Offline and optional.** The default build has no model. An optional lane may call a local model with a fixed seed and temperature zero; its proposals are recorded in the receipt (E3) so that a result can be replayed without the model.
4. **Start where the payoff is largest.** Carrier and helper proposals (R2, R5): one call per query, outside the inner loop, with the largest reduction of search space per call.

13.4 Questions for experts

1. What accuracy do current models reach at proposing the accumulator or auxiliary function for textbook recursive programs when given only the type and examples? A small study on the E8 suite would answer it.
2. Is there a way to make a model’s contribution deterministic and auditable enough for a tool whose output is cited in proofs of correctness, beyond recording proposals in receipts?

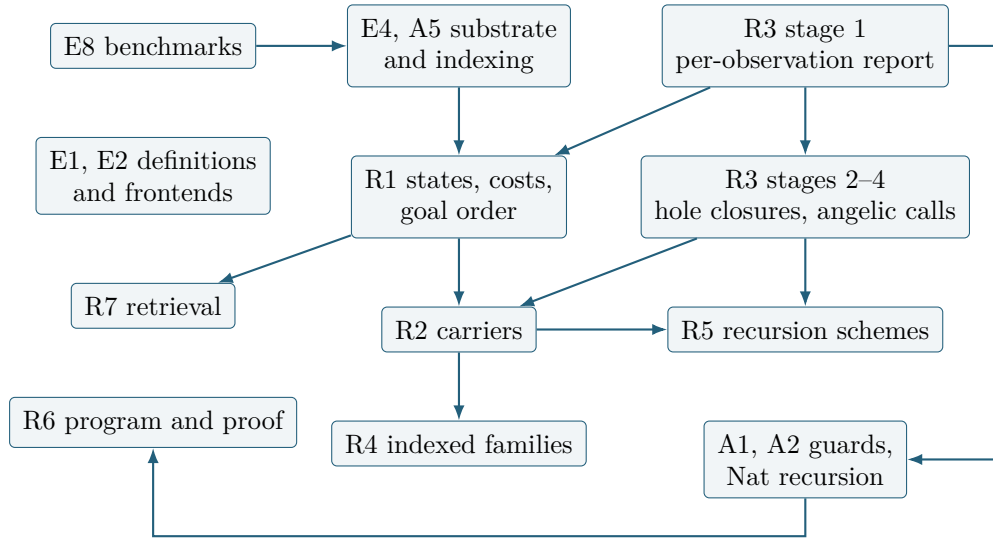
Related projects and fields: LeanDojo and the Lean hammer study retrieval and learned guidance for Lean; neurosymbolic program synthesis studies learned guidance of enumerative search.

Part IV

Plan

14 Roadmap

14.1 Dependencies



14.2 Phases and gates

Every phase ends with the six harnesses at full score for 10 s and 5 s budgets, plus its own gate.

Phase	Content	Gate
P1	E8 benchmark port; E1 compilable results; E4 profiling and per-local caching; R3 stage 1 (per-observation report)	benchmark baseline recorded (expected: most recursion and conditional entries fail); probe 6 binds cleanly; node cost under 1 ms on Church probes
P2	R1 steps 1–2 (first-class states, cost order); A5 indexing; A3 normal-form checks; E5 parallel lanes	identical candidate lists to the depth-first engine on the whole corpus; no repeated passes; uninhabited queries answered in lane time
P3	A1 guards; A2 <code>Nat</code> recursion and literals; E7 portfolio and bounded instantiation; E6 propositional decision procedure	probes 4, 9, 10; conditional and arithmetic benchmark entries; propositional queries certified both ways
P4	R3 stages 2–3; R1 steps 3–4 (learned costs, goal order by domain size)	differential agreement with the kernel on every generated closed program; solved count at 1 s improves on P2
P5	R2 carrier grammar, carrier refutation, observation-table proposals; R5 capability-based bodies and schemes; R4 index-generalizing induction	stretch cases; accumulator and tupling entries; probe 8; probe 11

Phase	Content	Gate
P6	R6 program and proof together; R7 retrieval; R8 behavioral classes; E2 frontends; E3 receipts	quantified-contract suite; probes 14, 16; reference-solution rank reported for the whole suite
P7	R9 proposal interface with an optional model lane	measured accuracy of proposed carriers; results replayable without the model

P1 to P3 are engineering and adaptation and can be scheduled with confidence. P4 is mostly engineering with one validation risk (agreement with the kernel). P5 is where the research risk is concentrated: it should be preceded by the expert consultations of Section 15 and by paper prototypes on the thirteen stretch cases, and its gate should be read as a target, not a commitment.

14.3 Risks

Risk	Consequence	Mitigation
Cost-ordered search changes which candidates are found first	baseline outcomes stay the same but transcripts change; ranking complaints	gate P2 on identical candidate lists before enabling costs; change costs only with the ranking benchmark in hand
Custom evaluator disagrees with the kernel	unsound pruning, lost solutions	conservative fragment; differential testing on every generated program; kernel fallback
Carrier enumeration slows solved cases	regression in time, not in score	carriers enter only after the plain fold fails at the current cost bound; carrier refutation before search
Definition elaborator too slow for recursion candidates	P5 recursion unusable in 10 s	realize only candidates that pass the contract by interpretation
Mathlib-scale index too large or slow to load	retrieval lane unusable interactively	environment-fingerprinted on-disk index; reachability pre-pass limits providers to tens
Determinism lost under parallel lanes or learned costs	results not reproducible	rank after all lanes stop; learned costs are a pure function of the query's own search history

15 Where outside expertise would help

The questions of Part III are collected here in the order in which answers would change the plan most. For each we name the projects and fields whose research addresses the question, and what we would bring to a conversation, because a specialist's time is best spent on a small, concrete artifact rather than on a description.

#	Question	Projects and fields that study it	What we bring
1	Given finitely many input-output pairs of a list function, compute a minimal fold carrier from a type grammar, or show none exists (R2)	program calculation (folds, tupling, the third homomorphism theorem); automata learning (minimal state spaces from observations)	the thirteen stretch cases with their observation tables; the fold-closure test worked out on reverse , foldl1 , maximum
2	Search organization for goals that share metavariables, other than copying or sequentializing (R1)	Aesop (best-first proof search in Lean); Canonical (refinement order in dependent inhabitation)	traces showing goal-order sensitivity; the deferral rules and the cases each was written for
3	Live bidirectional evaluation for higher-order examples and dependent types; a conservative fast evaluator relative to the Lean kernel (R3)	Hazel (hole closures); Smyth (bidirectional evaluation); Lean4Lean and the Lean kernel (reduction rules and their specification)	profile data; the per-observation decomposition; a corpus of 10^5 generated closed programs for differential testing
4	Enumerating motives and generalizations; canonical treatment of transports in search (R4)	the Lean elaborator (motive computation); Canonical and sauto (dependent inhabitation); Agsy and Mimer (term search in Agda); higher-order unification	probe 8 and ten further indexed-family problems with the motives written out
5	Evaluating and realizing capability-based recursive bodies; angelic execution in top-down search (R5)	Lean’s definition elaborator (structural and well-founded recursion); Burst and Trio (recursive synthesis from incomplete examples)	the capability design; measurements of definition-elaboration cost per candidate
6	Interleaving value holes and proof holes; the class of properties provable by induction along the program; state of automated induction in Lean (R6)	Synquid (refinement-type synthesis); grind , Aesop and the Lean hammer (proof automation); inductive theorem proving	the quantified-contract suite with hand proofs classified by the automation they need
7	Type abstraction for reachability over dependent, class-polymorphic libraries; relevance for programs rather than proofs (R7)	Hoogle+ and SyPet (component-based synthesis); exact? , Loogle and Lean premise selection (library lookup)	the head-level reachability graph of Lean core with its connectivity statistics
8	A formal objective for ranking inhabitants; what proof-assistant users expect (R8)	programming by example (ranking); human-factors research on proof assistants	the ranking benchmark with reference ranks under the current keys
9	Accuracy of model-proposed carriers and helpers; auditability (R9)	LeanDojo and the Lean hammer (learned retrieval); neurosymbolic program synthesis	the E8 suite as an evaluation set

Venues. The Lean community chat is the natural first venue for questions 2, 4, 5 and 6, which concern Lean’s own elaborator, kernel and automation. The others are questions for the program-synthesis and programming-languages research communities, reachable through the projects’ public repositories and the cited papers.

Order of contact. Question 1 first, because it decides whether R2 is a research project or an engineering one, and everything in P5 waits on it. Questions 2 and 3 next, because they shape the P2 and P4 rewrites of the search core and the evaluator, which are expensive to redo. The others can wait for their phases.

What we do not need help with. Parts I and II. The items there are understood, their tests are written down, and the measurements of Section 1 say in which order to do them.

References

- [1] A. Abel and B. Pientka. Higher-order dynamic pattern unification for dependent types and records. *TLCA*, 2011.
- [2] J. Limperg and A. H. From. Aesop: white-box best-first proof search for Lean. *CPP*, 2023.
- [3] F. Lindblad and M. Benke. A tool for automated theorem proving in Agda. *TYPES 2004*, LNCS 3839, 2006.
- [4] J.-P. Bernardy, P. Jansson and K. Claessen. Testing polymorphic properties. *ESOP*, 2010.
- [5] A. Miltner, A. T. Nuñez, A. Brendel, S. Chaudhuri and I. Dillig. Bottom-up synthesis of recursive functional programs using angelic execution. *POPL*, 2022.
- [6] C. Norman and J. Avigad. Canonical for automated theorem proving in Lean. *ITP*, 2025.
- [7] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia and V. Saraswat. Combinatorial sketching for finite programs. *ASPLOS*, 2006.
- [8] J. Cockx and D. Devriese. Proof-relevant unification: dependent pattern matching with only the axioms of your type theory. *Journal of Functional Programming*, 2018.
- [9] M. Balog, A. Gaunt, M. Brockschmidt, S. Nowozin and D. Tarlow. DeepCoder: learning to write programs. *ICLR*, 2017.
- [10] K. Ellis et al. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI*, 2021.
- [11] R. Dyckhoff. Contraction-free sequent calculi for intuitionistic logic. *Journal of Symbolic Logic*, 57(3), 1992.
- [12] A. Albarghouthi, S. Gulwani and Z. Kincaid. Recursive program synthesis. *CAV*, 2013.
- [13] R. Alur, A. Radhakrishna and A. Udupa. Scaling enumerative program synthesis via divide and conquer. *TACAS*, 2017.
- [14] J. Frankle, P.-M. Osera, D. Walker and S. Zdancewic. Example-directed synthesis: a type-theoretic interpretation. *POPL*, 2016.
- [15] C. Omar, I. Voysey, R. Chugh and M. Hammer. Live functional programming with typed holes. *POPL*, 2019.
- [16] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala and N. Polikarpova. Program synthesis by type-guided abstraction refinement. *POPL*, 2020.

- [17] G. Huet. A unification algorithm for typed λ -calculus. *Theoretical Computer Science*, 1(1), 1975.
- [18] R. Korf. Depth-first iterative-deepening: an optimal admissible tree search. *Artificial Intelligence*, 27(1), 1985.
- [19] J. Feser, S. Chaudhuri and I. Dillig. Synthesizing data structure transformations from input-output examples. *PLDI*, 2015.
- [20] K. Yang et al. LeanDojo: theorem proving with retrieval-augmented language models. *NeurIPS*, 2023.
- [21] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang and S. Welleck. Premise selection for a Lean hammer. Preprint, 2025.
- [22] M. Luby, A. Sinclair and D. Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4), 1993.
- [23] D. Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4), 1991.
- [24] P.-M. Osera and S. Zdancewic. Type-and-example-directed program synthesis. *PLDI*, 2015.
- [25] S. Barke, H. Peleg and N. Polikarpova. Just-in-time learning for bottom-up enumerative synthesis. *OOPSLA*, 2020.
- [26] Ł. Czajka. Practical proof search for Coq by type inhabitation. *IJCAR*, 2020.
- [27] C. S. Lee, N. Jones and A. Ben-Amram. The size-change principle for program termination. *POPL*, 2001.
- [28] J. Lubin, N. Collins, C. Omar and R. Chugh. Program sketching with live bidirectional evaluation. *ICFP*, 2020.
- [29] R. Statman. Intuitionistic propositional logic is polynomial-space complete. *Theoretical Computer Science*, 9(1), 1979.
- [30] N. Polikarpova, I. Kuraj and A. Solar-Lezama. Program synthesis from polymorphic refinement types. *PLDI*, 2016.
- [31] Y. Feng, R. Martins, Y. Wang, I. Dillig and T. Reps. Component-based synthesis for complex APIs. *POPL*, 2017.
- [32] J. Gibbons. The third homomorphism theorem. *Journal of Functional Programming*, 6(4), 1996.
- [33] A. Udupa, A. Raghavan, J. Deshmukh, S. Mador-Haim, M. Martin and R. Alur. TRANSIT: specifying protocols with concolic snippets. *PLDI*, 2013.
- [34] W. Lee and H. Cho. Inductive synthesis of structurally recursive functional programs from non-recursive expressions. *POPL*, 2023.
- [35] Z. Hu, H. Iwasaki, M. Takeichi and A. Takano. Tupling calculation eliminates multiple data traversals. *ICFP*, 1997.
- [36] P. Wadler. Theorems for free! *FPCA*, 1989.
- [37] J. Gibbons, G. Hutton and T. Altenkirch. When is a function a fold or an unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001.